MODULE 3 – MySQL Assessment

The given dataset is opened in the MySQL workbench and the whole script is executed. The table Customer_churn is displayed using SELECT.

```
5661 •  SELECT * FROM customer_churn;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

CustomerID	Churn	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome	PreferredPaymentMode	Gender	HourSpendOnApp	NumberOfDev
50001	1	4	Mobile Phone	3	6	Debit Card	Female	3	3
50002	1	NULL	Phone	1	8	UPI	Male	3	4
50003	1	NULL	Phone	1	30	Debit Card	Male	2	4
50004	1	0	Phone	3	15	Debit Card	Male	2	4
50005	1	0	Phone	1	12	CC	Male	NULL	3
50006	1	0	Computer	1	22	Debit Card	Female	3	5
50007	1	NULL	Phone	3	11	Cash on Delivery	Male	2	3
50008	1	NULL	Phone	1	6	CC	Male	3	3
50009	1	13	Phone	3	9	E wallet	Male	NULL	4
50010	1	NULL	Phone	1	31	Debit Card	Male	2	5
50011	1	4	Mobile Phone	1	18	Cash on Delivery	Female	2	3
50012	1	11	Mobile Phone	1	6	Debit Card	Male	3	4
50013	1	0	Phone	1	11	COD	Male	2	3
50014	1	0	Phone	1	15	CC	Male	3	4
50015	1	9	Mobile Phone	3	15	Credit Card	Male	3	4

Result Grid | Form Editor | Field Types | Query Stats

Data Cleaning

Handling missing values and outliers

- Imputed mean for the following columns WarehouseToHome, HourSpendOnApp, OrderAmountHikeFromlastYear, DaySinceLastOrder., and round off to the nearest integer using UPDATE and IFNULL. Since aggregate functions can be used in UPDATE statement. First a CTE was created to give the ROUND(AVG()) mean of each column an alias. This CTE is CROSSJOINed and UPDATEd together handle the null values.

The CTE created:

```
5665 •  WITH avg_values AS (
5666    SELECT
5667    ROUND(AVG(WarehouseToHome)) AS avg_warehouse,
5668    ROUND(AVG(HourSpendOnApp)) AS avg_app_hours,
5669    ROUND(AVG(OrderAmountHikeFromlastYear)) AS avg_hike,
5670    ROUND(AVG(DaySinceLastOrder)) AS avg_days_since_order
5671    FROM customer_churn
5672    )
5673    SELECT * FROM avg_values;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

avg_warehouse	avg_app_hours	avg_hike	avg_days_since_order
16	3	16	5

This CTE is cross joined and IFNULL is applied to fill all the null values in the selected columns

Before Imputing:

```
5662 •  SELECT CustomerID,
5663    WarehouseToHome,
5664    HourSpendOnApp,
5665    OrderAmountHikeFromlastYear,
5666    DaySinceLastOrder FROM customer_churn;
5667
5668 •  SET SQL_SAFE_UPDATES = 0;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

CustomerID	WarehouseToHome	HourSpendOnApp	OrderAmountHikeFromlastYear	DaySinceLastOrder
50001	6	3	11	5
50002	8	3	15	0
50003	30	2	14	3
50004	15	2	23	3
50005	12	NULL	11	3
50006	22	3	22	7
50007	11	2	14	0
50008	6	3	16	0
50009	9	NULL	14	2
50010	31	2	12	1
50011	18	2	NULL	8
50012	6	3	13	0
50013	11	2	13	2

After Imputing:

```
5670  WITH avg_values AS (
5671  SELECT
5672  ROUND(AVG(WarehouseToHome)) AS avg_warehouse,
5673  ROUND(AVG(HourSpendOnApp)) AS avg_app_hours,
5674  ROUND(AVG(OrderAmountHikeFromlastYear)) AS avg_hike,
5675  ROUND(AVG(DaySinceLastOrder)) AS avg_days_since_order
5676  FROM customer_churn
5677  )
5678  UPDATE customer_churn c
5679  CROSS JOIN avg_values a
5680  SET
5681  c.WarehouseToHome = IFNULL(c.WarehouseToHome, a.avg_warehouse),
5682  c.HourSpendOnApp = IFNULL(c.HourSpendOnApp, a.avg_app_hours),
5683  c.OrderAmountHikeFromlastYear = IFNULL(c.OrderAmountHikeFromlastYear, a.avg_hike),
5684  c.DaySinceLastOrder = IFNULL(c.DaySinceLastOrder, a.avg_days_since_order)
5685  WHERE
5686  c.WarehouseToHome IS NULL
5687  OR c.HourSpendOnApp IS NULL
5688  OR c.OrderAmountHikeFromlastYear IS NULL
5689  OR c.DaySinceLastOrder IS NULL;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

CustomerID	WarehouseToHome	HourSpendOnApp	OrderAmountHikeFromlastYear	DaySinceLastOrder
50006	22	3	22	7
50007	11	2	14	0
50008	6	3	16	0
50009	9	3	14	2
50010	31	2	12	1

customer_churn 3 | Apply

- Same method as earlier is used. Instead of CTE of avg_values, a CTE of mode values is made and CROSSJOINed to the customer_churn table and use IFNULL and UPDATE to fill the null with imputed Mode

CTE created for mode:

```
5687  WITH mode_values AS (
5688  SELECT
5689  (SELECT Tenure FROM customer_churn
5690  WHERE Tenure IS NOT NULL GROUP BY Tenure
5691  ORDER BY COUNT(*) DESC LIMIT 1) AS mode_tenure,
5692  (SELECT CouponUsed FROM customer_churn
5693  WHERE CouponUsed IS NOT NULL GROUP BY CouponUsed
5694  ORDER BY COUNT(*) DESC LIMIT 1) AS mode_CouponUsed,
5695  (SELECT OrderCount FROM customer_churn
5696  WHERE OrderCount IS NOT NULL GROUP BY OrderCount
5697  ORDER BY COUNT(*) DESC LIMIT 1) AS mode_count
5698  )
5699  SELECT * FROM mode_values;
5700
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

mode_tenure	mode_CouponUsed	mode_count
1	1	2

Before Imputing:

Result Grid | Filter Rows: | Export: | Wrap Cell Con

CustomerID	Tenure	CouponUsed	OrderCount
50001	4	1	1
50002	NULL	0	1
50003	NULL	0	1
50004	0	0	1
50005	0	1	1
50006	0	4	6
50007	NULL	0	1
50008	NULL	2	2
50009	13	0	1
50010	NULL	1	1
50011	4	9	15
50012	11	0	1

customer_churn 7 ×

Output

After Imputing:

```
5691  WITH mode_values AS (
5692  SELECT
5693  (SELECT Tenure FROM customer_churn
5694  WHERE Tenure IS NOT NULL GROUP BY Tenure
5695  ORDER BY COUNT(*) DESC LIMIT 1) AS mode_tenure,
5696  (SELECT CouponUsed FROM customer_churn
5697  WHERE CouponUsed IS NOT NULL GROUP BY CouponUsed
5698  ORDER BY COUNT(*) DESC LIMIT 1) AS mode_CouponUsed,
5699  (SELECT OrderCount FROM customer_churn
5700  WHERE OrderCount IS NOT NULL GROUP BY OrderCount
5701  ORDER BY COUNT(*) DESC LIMIT 1) AS mode_count
5702  )
5703  UPDATE customer_churn c
5704  CROSS JOIN mode_values m SET
5705  c.Tenure = IFNULL(c.Tenure, m.mode_tenure),
5706  c.CouponUsed = IFNULL(c.CouponUsed, m.mode_CouponUsed),
5707  c.OrderCount = IFNULL(c.OrderCount, m.mode_count)
5708  WHERE c.Tenure IS NULL
5709  OR c.CouponUsed IS NULL
5710  OR c.OrderCount IS NULL;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

CustomerID	Tenure	CouponUsed	OrderCount
50001	4	1	1
50002	1	0	1
50003	1	0	1
50004	0	0	1
50005	0	1	1

- There are two rows in customer_churn table that has ‘WarehouseToHome greater than 100

```
5713  SELECT * FROm customer_churn WHERE WarehouseToHome > 100;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerID	Churn	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome	PreferredPaymentMode	Gender	HourSpendOnApp	NumberOfDevice
51310	0	25	Computer	3	126	Debit Card	Male	2	3
54125	0	26	Computer	3	127	Debit Card	Male	3	4

DELETE statement is used to delete these rows.

```
5715 •    DELETE FROM customer_churn WHERE WarehouseToHome > 100;
```

Output

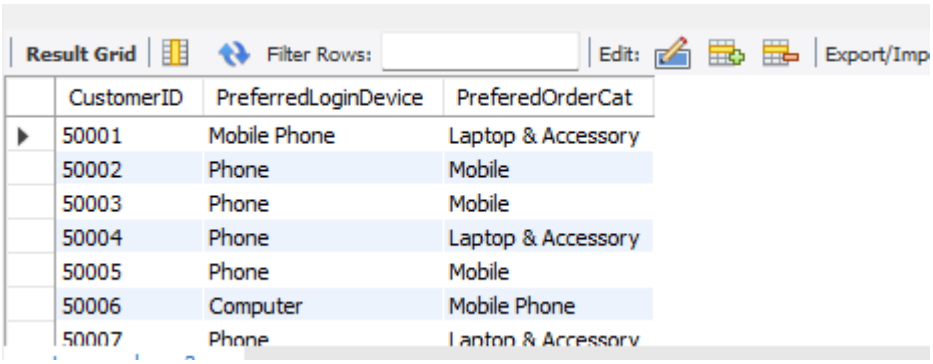
Action Output

#	Time	Action	Message
34	14:42:59	SELECT CustomerID,Tenure,CouponUsed,OrderCount FROM customer_churn	5630 row(s) returned
35	14:51:04	SELECT * FROm customer_churn WHERE WarehouseToHome > 100	2 row(s) returned
36	14:53:55	DELETE FROM customer_churn WHERE WarehouseToHome > 100	2 row(s) affected

Dealing with Inconsistencies:

- Replaced occurrences of “Phone” in the 'PreferredLoginDevice' column and “Mobile” in the 'PreferedOrderCat' column with “Mobile Phone” to ensure uniformity. UPDATE command is used to do this.

Before Replacing:

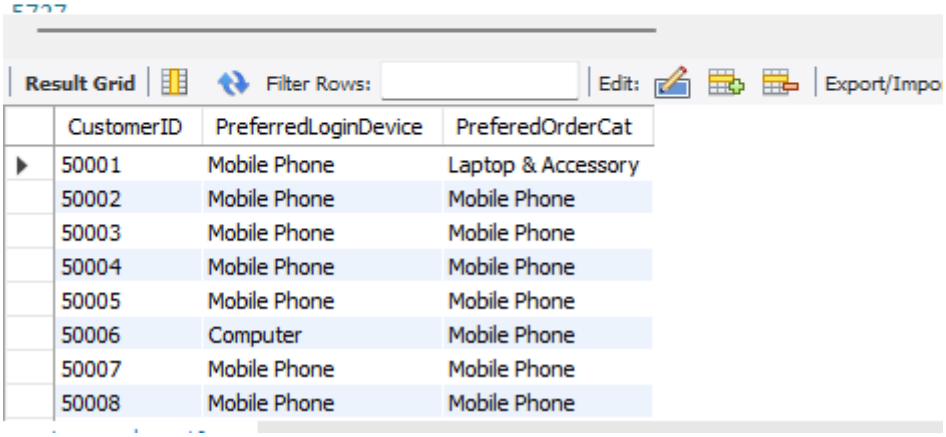


Result Grid | Filter Rows: | Edit: | Export/Imp

CustomerID	PreferredLoginDevice	PreferedOrderCat
50001	Mobile Phone	Laptop & Accessory
50002	Phone	Mobile
50003	Phone	Mobile
50004	Phone	Laptop & Accessory
50005	Phone	Mobile
50006	Computer	Mobile Phone
50007	Phone	Laptop & Accessory

After Replacing:

```
5720 •    UPDATE customer_churn
5721      SET
5722      PreferredLoginDevice = 'Mobile Phone',
5723      PreferedOrderCat = 'Mobile Phone'
5724      WHERE
5725      PreferredLoginDevice = 'Phone'
5726      OR PreferedOrderCat = 'Mobile';
```



Result Grid | Filter Rows: | Edit: | Export/Impo

CustomerID	PreferredLoginDevice	PreferedOrderCat
50001	Mobile Phone	Laptop & Accessory
50002	Mobile Phone	Mobile Phone
50003	Mobile Phone	Mobile Phone
50004	Mobile Phone	Mobile Phone
50005	Mobile Phone	Mobile Phone
50006	Computer	Mobile Phone
50007	Mobile Phone	Mobile Phone
50008	Mobile Phone	Mobile Phone

- Standardize payment mode values: Replaced "COD" with "Cash on Delivery" and "CC" with "Credit Card" in the PreferredPaymentMode column. We use CASE statement to put in conditions to the column.

Before Replacing:

```
5728 •   SELECT CustomerID, PreferredPaymentMode FROM customer_churn;
```

Result Grid | Filter Rows: | Edit: | Export/Import:

CustomerID	PreferredPaymentMode
50011	Cash on Delivery
50012	Debit Card
50013	COD
50014	CC
50015	Credit Card
50016	UPI
50017	Debit Card
50018	E wallet

After Replacing:

```
5730 •   UPDATE customer_churn
5731     SET
5732     PreferredPaymentMode = CASE PreferredPaymentMode
5733     WHEN 'CC' THEN 'Credit Card'
5734     WHEN 'COD' THEN 'Cash on Delivery'
5735     ELSE PreferredPaymentMode
5736     END
5737     WHERE PreferredPaymentMode IN('CC','COD');
5738
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

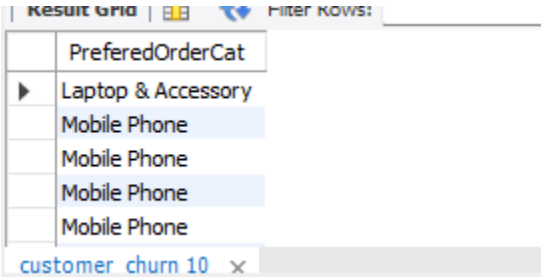
CustomerID	PreferredPaymentMode
50010	Debit Card
50011	Cash on Delivery
50012	Debit Card
50013	Cash on Delivery
50014	Credit Card
50015	Credit Card

Data Transformation:

Column Renaming:

- Renamed the column "PreferedOrderCat" to "PreferredOrderCat". We use ALTER and RENAME command to rename the column.

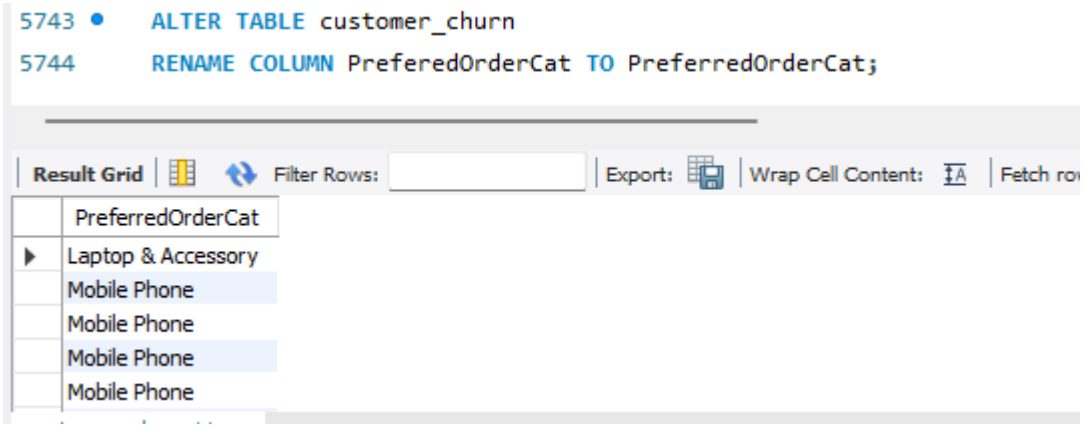
Before renaming:



	PreferedOrderCat
▶	Laptop & Accessory
	Mobile Phone
	Mobile Phone
	Mobile Phone
	Mobile Phone

customer_churn 10

After renaming:



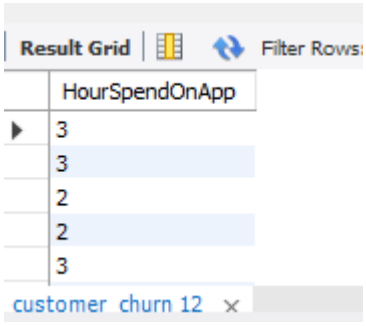
```
5743    ALTER TABLE customer_churn
5744    RENAME COLUMN PreferedOrderCat TO PreferredOrderCat;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch ro

	PreferredOrderCat
▶	Laptop & Accessory
	Mobile Phone
	Mobile Phone
	Mobile Phone
	Mobile Phone

- Renamed the column "HourSpendOnApp" to "HoursSpentOnApp" with same method as above.

Before renaming:



Result Grid | Filter Rows:

	HourSpendOnApp
▶	3
	3
	2
	2
	3

customer_churn 12

After renaming:

```
5748 •  ALTER TABLE customer_churn
5749    RENAME COLUMN HourSpendOnApp TO HoursSpentOnApp;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

HoursSpentOnApp
3
3
2
2
3

Creating New Columns:

- Created a new column named ‘ComplaintReceived’ with values "Yes" if the corresponding value in the ‘Complain’ is 1, and "No" otherwise.
  First a column is added using ALTER and ADD COLUMN in to the table.
  Then we will use a CASE statement to fill the column using IF else conditions.

Column Creation:

```
5753 •  ALTER TABLE customer_churn
5754    ADD COLUMN ComplaintReceived VARCHAR(10);
5755
5756 •  SELECT Complain, ComplaintReceived FROM customer_churn;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Complain	ComplaintReceived
1	NULL
1	NULL
1	NULL
0	NULL
0	NULL

CASE statement:

```
5758 •  UPDATE customer_churn
5759    SET ComplaintReceived = CASE
5760        WHEN Complain = 1 THEN 'Yes'
5761        ELSE 'No'
5762    END;
```

Result Grid | Filter Rows: | Export:

Complain	ComplaintReceived
1	Yes
1	Yes
1	Yes
0	No
0	No

- Created a new column named 'ChurnStatus'. Set its value to “Churned” if the corresponding value in the 'Churn' column is 1, else assign “Active”. Same method as above is used.

Column Creation:

```
5764    ALTER TABLE customer_churn
5765    ADD COLUMN ChurnStatus VARCHAR(20);
5766
5767    SELECT Churn,ChurnStatus FROM customer_churn;
```

Result Grid | Filter Rows: | Export: | Wrap

Churn	ChurnStatus
1	NULL
1	NULL
1	NULL
1	NULL
1	NULL

CASE statement:

```
5769    UPDATE customer_churn
5770    SET ChurnStatus = CASE
5771        WHEN Churn = 1 THEN 'Churned'
5772        ELSE 'Active'
5773    END;
5774
```

Result Grid | Filter Rows: | Export:

Churn	ChurnStatus
0	Active
1	Churned
0	Active
0	Active
0	Active

Column Dropping:

- Dropped the columns "Churn" and "Complain" from the table.

Before Dropping:

```
5776    SELECT Churn,Complain FROM customer_churn;
```

Result Grid | Filter Rows: | Export: | Wrap Cell

Churn	Complain
1	1
1	1
1	1
1	0
1	0

After Dropping:

```
5778    ALTER TABLE customer_churn
5779    DROP COLUMN Churn,
5780    DROP COLUMN Complain;
```

Output

Action Output

#	Time	Action
48	11:29:48	SELECT Churn,Complain FROM customer_churn
49	11:33:14	ALTER TABLE customer_churn DROP COLUMN Churn, DROP COLUMN C
50	11:33:53	SELECT Churn,Complain FROM customer_churn

Data Exploration and Analysis:

- Retrieved the count of churned and active customers from the dataset.

```
5782    SELECT ChurnStatus,Count(*) FROM customer_churn GROUP BY ChurnStatus;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ChurnStatus	Count(*)
Churned	948
Active	4680

- Displayed the average tenure and total cashback amount of customers who churned.

```
5786    SELECT ChurnStatus, AVG(Tenure) AS Avg_Tenure,
5787    SUM(CashbackAmount) AS Total_Cashback
5788    FROM customer_churn
5789    GROUP BY ChurnStatus HAVING ChurnStatus = 'Churned';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

ChurnStatus	Avg_Tenure	Total_Cashback
Churned	3.1762	152030

- Determined the percentage of churned customers who complained.

```
5791    SELECT
5792    ((SELECT COUNT(*) FROM customer_churn
5793    WHERE ChurnStatus = 'Churned'
5794    AND ComplaintReceived = 'Yes')/
5795    (SELECT COUNT(*) FROM customer_churn
5796    WHERE ChurnStatus = 'Churned') * 100)
5797    AS Percentage_Churned_Who_Complained;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Percentage_Churned_Who_Complained
53.5865

- Identified the city tier with the highest number of churned customers whose preferred order category is Laptop & Accessory.

```
5799    SELECT CityTier, COUNT(*) AS Churned_customers FROM customer_churn
5800    WHERE ChurnStatus = 'Churned'
5801    AND PreferredOrderCat = 'Laptop & Accessory'
5802    GROUP BY CityTier
5803    ORDER BY Churned_customers DESC
5804    LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CityTier	Churned_customers
3	137

- Identifed the most preferred payment mode among active customers.

```
5806    SELECT PreferredPaymentMode,ChurnStatus,COUNT(*)
5807    AS No_of_Payments FROM customer_churn
5808    WHERE ChurnStatus = 'Active'
5809    GROUP BY PreferredPaymentMode
5810    ORDER BY No_of_Payments DESC
5811    LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

PreferredPaymentMode	ChurnStatus	No_of_Payments
Debit Card	Active	1956

- Calculate the total order amount hike from last year for customers who are single and prefer mobile phones for ordering.

```
5813    SELECT PreferredLoginDevice, MaritalStatus, SUM(OrderAmountHikeFromLastYear)
5814    AS Total_Amount_Hike_From_Last_Year
5815    FROM customer_churn
5816    WHERE MaritalStatus = 'Single'
5817    GROUP BY PreferredLoginDevice HAVING PreferredLoginDevice = 'Mobile Phone';
5818
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

PreferredLoginDevice	MaritalStatus	Total_Amount_Hike_From_Last_Year
Mobile Phone	Single	20798

- Find the average number of devices registered among customers who used UPI as their preferred payment mode.

```
5821    SELECT PreferredPaymentMode, ROUND(AVG(NumberOfDeviceRegistered),0)
5822    AS Avg_No_Devices_Registered
5823    FROM customer_churn
5824    GROUP BY PreferredPaymentMode HAVING PreferredPaymentMode = 'UPI';
5825
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

PreferredPaymentMode	Avg_No_Devices_Registered
UPI	4

- Determine the city tier with the highest number of customers.

```
5827    SELECT CityTier, COUNT(CustomerID) AS No_of_Customers
5828    FROM customer_churn
5829    GROUP BY CityTier
5830    ORDER BY No_of_Customers DESC
5831    LIMIT 1;
5832
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CityTier	No_of_Customers
1	3666

- Identify the gender that utilized the highest number of coupons.

```
5834 •  SELECT Gender, SUM(CouponUsed) AS No_of_Coupons_Used
5835    FROM customer_churn
5836    GROUP BY Gender
5837    ORDER BY No_of_Coupons_Used DESC
5838    LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Gender	No_of_Coupons_Used
Male	5629

- List the number of customers and the maximum hours spent on the app in each preferred order category.

```
5841 •  SELECT PreferredOrderCat, COUNT(*) AS No_of_Customers,
5842    MAX(HoursSpentOnApp) AS Max_Hours_Spent_On_App
5843    FROM customer_churn
5844    GROUP BY PreferredOrderCat;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

PreferredOrderCat	No_of_Customers	Max_Hours_Spent_On_App
Laptop & Accessory	1785	5
Mobile Phone	2344	5
Others	264	4
Fashion	825	5
Grocery	410	4

- Calculate the total order count for customers who prefer using credit cards and have the maximum satisfaction score.

```
5847 •  SELECT PreferredPaymentMode, COUNT(*) AS No_of_customers,
5848    MAX(SatisfactionScore) AS Maximum_Satisfaction_Score
5849    FROM customer_churn
5850    GROUP BY PreferredPaymentMode
5851    HAVING PreferredPaymentMode = 'Credit Card';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

PreferredPaymentMode	No_of_customers	Maximum_Satisfaction_Score
Credit Card	1774	5

- What is the average satisfaction score of customers who have complained?

```
5854 •  SELECT AVG(SatisfactionScore) AS Avg_Satsfaction_Score
5855    FROM customer_churn
5856    WHERE ComplaintReceived = 'Yes';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Avg_Satsfaction_Score
2.9988

- List the preferred order category among customers who used more than 5 coupons.

```
5859 •  SELECT PreferredOrderCat, COUNT(CouponUsed) AS No_of_Customers_Who_Used_More_Than_5
5860    FROM customer_churn
5861    WHERE CouponUsed >= 5
5862    GROUP BY PreferredOrderCat
5863    ORDER BY No_of_Customers_Who_Used_More_Than_5 DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

PreferredOrderCat	No_of_Customers_Who_Used_More_Than_5
Laptop & Accessory	132
Fashion	126
Mobile Phone	85
Grocery	50
Others	39

- List the top 3 preferred order categories with the highest average cashback amount.

```
5866 •  SELECT PreferredOrderCat, AVG(CashbackAmount) AS Avg_Cashback_Amount
5867    FROM customer_churn
5868    GROUP BY PreferredOrderCat
5869    ORDER BY Avg_Cashback_Amount DESC
5870    LIMIT 3;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

PreferredOrderCat	Avg_Cashback_Amount
Others	304.4545
Grocery	266.2366
Fashion	210.4824

- Find the preferred payment modes of customers whose average tenure is 10 months and have placed more than 500 orders. (I cant find any payment mode with average exactly at 10 months)

```sql
SELECT PreferredPaymentMode, AVG(Tenure) AS Avg_tenure, SUM(OrderCount) AS No_of_orders
FROM customer_churn
GROUP BY PreferredPaymentMode
HAVING Avg_tenure > 10
AND No_of_orders > 500;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

PreferredPaymentMode	Avg_tenure	No_of_orders
E wallet	10.1954	1849

- Categorize customers based on their distance from the warehouse to home such as 'Very Close Distance' for distances <=5km, 'Close Distance' for <=10km, 'Moderate Distance' for <=15km, and 'Far Distance' for >15km. Then, display the churn status breakdown for each distance category.

```sql
SELECT
CASE
WHEN WarehouseToHome <= 5 THEN 'Very Close Distance'
WHEN WarehouseToHome <= 10 THEN 'Close Distance'
WHEN WarehouseToHome <= 15 THEN 'Moderate Distance'
ELSE 'Far Distance'
END AS Distance_level,
ChurnStatus, COUNT(*) AS No_of_customers
FROM customer_churn
GROUP BY Distance_level,ChurnStatus
ORDER BY Distance_level,ChurnStatus;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Distance_level	ChurnStatus	No_of_customers
Close Distance	Active	1696
Close Distance	Churned	265
Far Distance	Active	1871
Far Distance	Churned	498
Moderate Distance	Active	1106
Moderate Distance	Churned	184
Very Close Distance	Active	7

- List the customer’s order details who are married, live in City Tier-1, and their order counts are more than the average number of orders placed by all customers.

```
5893 •  SELECT * FROM customer_churn
5894    WHERE MaritalStatus = 'Married'
5895    AND CityTier = 1
5896    AND OrderCount > (SELECT AVG(OrderCount) FROM customer_churn);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

CustomerID	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome	PreferredPaymentMode	Gender	HoursSpentOnApp	Numbe
50049	3	Computer	1	15	Credit Card	Male	3	3
50119	13	Mobile Phone	1	30	Cash on Delivery	Male	3	3
50132	15	Mobile Phone	1	16	Credit Card	Female	2	4
50133	13	Mobile Phone	1	8	Credit Card	Male	2	3
50152	2	Mobile Phone	1	28	Debit Card	Female	3	2
50180	12	Mobile Phone	1	6	Credit Card	Male	2	2

- a)

```
5899 •  CREATE TABLE customer_returns(
5900      Return_ID INT Primary key,
5901      Customer_ID INT,
5902      Return_Date Date,
5903      Refund_Amount INT
5904      );
5905 •  SELECT * FROM customer_returns;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Return_ID	Customer_ID	Return_Date	Refund_Amount

```
5908 •  INSERT INTO customer_returns VALUES
5909    (1001, 50022, '2023-01-01', 2130),
5910    (1002, 50316, '2023-01-23', 2000),
5911    (1003, 51099, '2023-02-14', 2290),
5912    (1004, 52321, '2023-03-08', 2510),
5913    (1005, 52928, '2023-03-20', 3000),
5914    (1006, 53749, '2023-04-17', 1740),
5915    (1007, 54206, '2023-04-21', 3250),
5916    (1008, 54838, '2023-04-30', 1990);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content

Return_ID	Customer_ID	Return_Date	Refund_Amount
1001	50022	2023-01-01	2130
1002	50316	2023-01-23	2000
1003	51099	2023-02-14	2290
1004	52321	2023-03-08	2510
1005	52928	2023-03-20	3000

b)

```
5920    SELECT r.Return_ID, r.Return_Date, r.Refund_Amount, c.*
5921    FROM customer_returns r
5922    INNER JOIN customer_churn c
5923    ON r.Customer_ID = c.CustomerID
5924    WHERE c.ChurnStatus = 'Churned'
5925    AND ComplaintReceived = 'Yes';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Return_ID	Return_Date	Refund_Amount	CustomerID	Tenure	PreferredLoginDevice	CityTier	WarehouseToHome
1002	2023-01-23	2000	50316	0	Computer	2	29
1004	2023-03-08	2510	52321	18	Mobile Phone	3	19
1006	2023-04-17	1740	53749	1	Mobile Phone	3	31